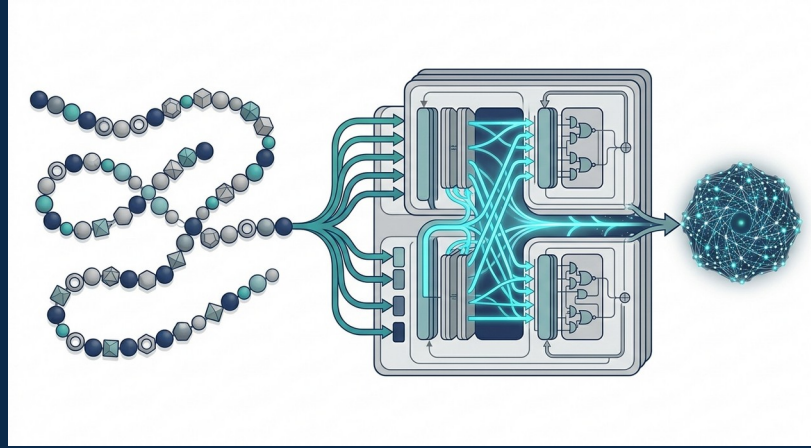




## 3교시 · 학습된 표현 & 구조 기반 표현

기계가 표현을 '학습'한다(PLM) + 구조를 표현으로 → 비교하고 실습

7/24 강의 3·4교시 통합·정리 · 마지막 교시 · 재직자·전공자 · 한국어

## 이 교시가 끝나면

- 단백질 언어모델(PLM)이 표현을 '학습'하는 원리(MLM)를 설명한다  
ESM-2 · ProtT5
- 서열 임베딩을 얻어 downstream에 활용하는 법을 안다  
실습: ESM-2 임베딩
- 구조 기반 표현(contact · graph · surface)을 구분한다
- AlphaFold/ESMFold를 '표현 소스'로 쓰는 법과 한계를 안다
- 데이터 · 과제 · 해석성 · 비용으로 표현을 선택하고 실측으로 비교한다

## 사람이 '설계'하던 feature → 기계가 '학습'하는 표현

### 2교시 — 수작업 표현

- 사람이 feature를 직접 설계
- one-hot·조성(AAC)·물성(ProtParam)
- 진화정보 PSSM/MSA·예측 툴
- 투명·해석적, 그러나 정보 상한 존재

### 3교시 — 학습된·구조 표현

- 대규모 사전학습이 표현을 대신 학습
- PLM 임베딩(ESM-2·ProtT5)
- 구조를 표현으로(contact·graph·surface)
- AlphaFold를 표현 소스로 → 비교·실습

#### +심화

핵심 전환: 2교시는 '무엇을 feature로 쓸지'를 사람이 정했다. 3교시는 수억 서열의 사전학습이 그 feature 설계를 대신한다 — 표현을 설계에서 학습으로 옮기는 것이 오늘의 이야기.

## 이 교시에서 볼 것

### A. 단백질 언어모델(PLM)

MLM · ESM-2 · ProtT5 · 임베딩 · 전이학습(+실습)

### B. 구조 기반 표현

contact · graph · surface · voxel

### C. 구조 예측을 소스로

AlphaFold2 · ESMFold · pLDDT · 한계 · SBDD

### D. 비교 · 선택

선택 매트릭스 · 벤치마크 · 한계

### E. 오늘의 실습

20개 · 4 family로 표현 실측 비교

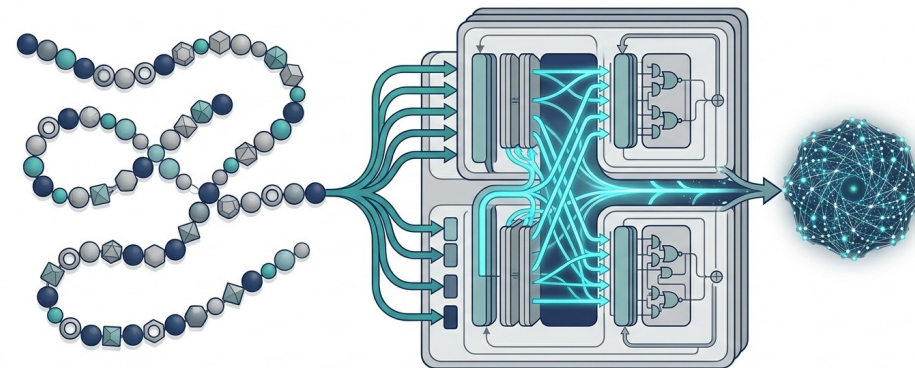
PART A

# 단백질 언어모델 (PLM)

표현을 '설계'하지 않고 '학습'한다

## 단백질 언어모델 — 무엇이 다른가

- 서열을 '언어'로 보고 대규모 자기지도 학습(라벨 없음)
- 수억~수십억 서열로 사전학습 → 문맥 반영 잔기 임베딩
- 수작업 feature(2교시)를 학습된 표현이 대체·보완
- 한 번 학습하면 다양한 downstream에 전이(transfer)



용어 · PLM

protein language model — 대규모 서열로 사전학습해 문맥적 표현을 만드는 모델(ESM·ProtT5 등)

## 수작업 feature vs 학습된 표현

### 수작업 (2교시)

- 사람이 feature 설계(구성·물성·MSA)
- 투명·해석·소규모에 강함
- 도메인 지식 필요·정보 상한 존재
- MSA는 검색 느림·고아 취약

### 학습된 표현 (PLM)

- 데이터에서 표현을 자동 학습
- 문맥·전이·대규모에 강함
- 블랙박스·계산비용·데이터 의존
- 추론 시 MSA 불필요

#### +심화

둘은 경쟁이자 보완 — 소규모·해석엔 수작업, 복잡·대규모엔 학습표현. 실무에선 PLM 임베딩 + 물성 몇 개를 함께 쓰는 하이브리드가 흔하다(뒤 PART D·E에서 비교).

## 단백질 표현의 3단계 진화

### ① 수작업 특성공학 (2000s~)

one-hot · 조성 · 물성(AAindex/ProtParam) — 사람이 feature 설계 + SVM/RF (2교시)

### ② 진화정보 (~2019)

PSSM · MSA · 공진화 — 동족체 많으면 강력, 그러나 MSA 검색 느림 · 고아 취약 (2교시)

### ③ 학습된 표현 PLM (2021~)

ESM · ProtT5 임베딩이 '새 표준 feature' — 대규모 사전학습, MSA 없이 단일서열

### ③+ 구조+PLM 융합 (2023~)

구조 인식 PLM · GNN · AlphaFold 소스 — 서열 임베딩에 3D 정보 결합

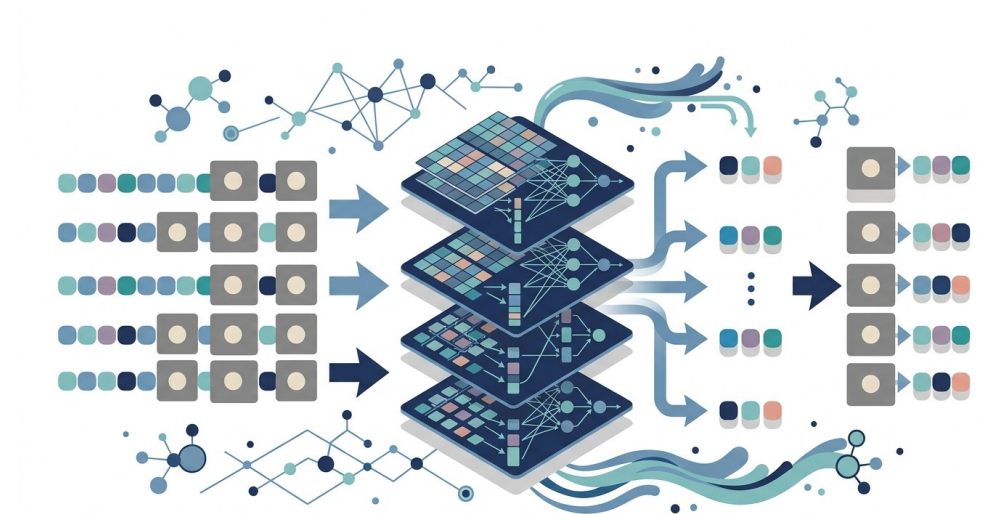
#### +심화

'특성공학이 사라진' 게 아니다 — PLM 임베딩이 새 표준 feature로 편입됐을 뿐, 물성 · 구조 보정은 해석 · 후처리로 여전히 생존. 3교시는 ③ · ③+ 를 다룬다.



## MLM — 빈칸 채우기로 표현을 배운다

- 서열의 ~15%를 가리고 원래 잔기를 예측
- 맞히려면 '구조·기능 제약'을 내부에 학습
- 공진화(함께 변하는 잔기쌍)가 표현에 흡수
- 라벨 없이 수억 서열로 자기지도 학습



용어 · MLM

masked language modeling — 가린 토큰 복원 목적함수  $L = -\sum \log p(x_i | \text{나머지})$ . BERT 계승

# MLM(masked language modeling) — PLM이 배우는 법

## 핵심 원리

1. **토큰화** — 각 아미노산=1토큰(+BOS/EOS 특수토큰)
2. **마스킹** — 무작위 15% 잔기 선택 → 80%[MASK]·10%랜덤·10%원본
3. **양방향 인코딩** — Transformer self-attention이 좌우 전체 문맥 반영
4. **복원 예측** — 마스크 위치 원래 아미노산을 20-way softmax로 맞힘(cross-entropy)
5. **대규모 반복** — UniRef 수억 서열 사전학습 → 문맥적 잔기 임베딩
6. **신호 창발** — 공변이·contact·2차구조가 라벨 없이 임베딩에 나타남

## 차별점 · 강점

- ▶ 라벨 없이 대규모 학습(PSSM은 계열별 정렬 필요)
- ▶ 양방향 문맥(자기회귀 LM은 한쪽만)
- ▶ 추론 시 MSA 불필요 — 단일 서열로 임베딩

## 한계 · 주의

- 대량 데이터·계산 필요
- 표현 내용 불투명(블랙박스)
- 서열에 없는 신호는 못 배움

# 주요 단백질 언어모델

| 모델                | 규모·계열                         | 메모(출처)                |
|-------------------|-------------------------------|-----------------------|
| ESM-1b / ESM-2    | 650M → 15B · Transformer(MLM) | 구조 창발 · ESMFold(2023) |
| ESM-C             | 표현학습 전용 계열                    | 임베딩 특화(2024, 기술공개)    |
| ProtBERT / ProtT5 | 420M / 3B · BERT · T5         | ProtTrans(2022)       |
| Ankh              | 효율형 T5                        | 적은 파라미터로 경쟁           |
| ProGen2           | 6.4B · 자기회귀                   | 생성 · fitness          |

+심화

임베딩 차원 예: ESM-2-650M · ESM-1b=1280, ProtBERT/ProtT5=1024, ESM-2-8M=320. per-residue(L×d) 또는 mean-pool로 per-protein(d)로 사용.

## ESM-2 — 스케일이 구조를 부른다

### 핵심 원리

1. **아키텍처** — Transformer 인코더 + rotary position embedding, 8M~15B 파라미터
2. **학습** — UniRef50/90 클러스터에서 MLM(마스크 복원)
3. **contact 추출** — attention map 대칭화 → 로지스틱 회귀로 잔기 contact(비지도)
4. **스케일 법칙** — 파라미터 ↑ → perplexity ↓ → 예측 구조 정확도 ↑ (구조 창발)
5. **ESMFold** — ESM-2 임베딩 위에 folding trunk → 단일서열 구조 예측
6. **활용** — 임베딩(예: 1280차원)이 downstream 표준 feature

### 차별점 · 강점

- ▶ MSA 불필요 — 단일서열로 임베딩·구조
- ▶ attention→contact가 스케일로 급향상
- ▶ 고아·설계 단백질(동족체 無)에 강함

### 한계 · 주의

- 15B는 GPU 메모리 큼
- MSA 풍부 타깃엔 AF2가 더 정확
- 단일 구조(양상불 아님)

# ProtTrans — BERT vs T5, 목적함수의 차이

## 핵심 원리

1. **ProtBERT** — BERT 인코더 + MLM(15% 마스크 복원), UniRef/BFD 학습
2. **ProtT5** — T5 인코더-디코더 + span corruption(연속 구간 마스크→복원)
3. **임베딩 사용** — ProtT5는 '인코더' 표현만 downstream에 사용(1024차원)
4. **학습 규모** — 대규모 아미노산 코퍼스 · TPU; ProtT5-XL 3B 파라미터
5. **단일서열** — 둘 다 추론 시 MSA 불필요 → 2차구조·국재화에 강함
6. **Ankh** — T5형 + 효율 설계로 적은 파라미터로 경쟁

## 차별점 · 강점

- ▶ BERT(MLM) vs T5(span corruption) 차이
- ▶ ProtT5 인코더 표현이 SS·localization서 강함
- ▶ MSA 없이 단일서열 임베딩

## 한계 · 주의

- 대형 모델 메모리·비용
- pooling·layer 선택에 민감

## SaProt / ProstT5 — 구조를 서열처럼 (A→B 다리)

### 핵심 원리

1. **동기** — 서열 PLM은 구조를 암묵적으로만 학습 → 명시 주입
2. **3Di 토큰** — Foldseek 구조 알파벳(20종)으로 각 잔기 국소구조 이산화
3. **SaProt** — (아미노산×3Di)=400 결합 vocabulary로 MLM 사전학습
4. **ProstT5** — 서열↔3Di 양방향 '번역'(T5 인코더-디코더)
5. **구조 공급** — 3Di는 AlphaFold/ESMFold 예측 구조에서 계산(PART C)
6. **효과** — fold-level 원거리 상동성까지 서열모델에 반영

### 차별점 · 강점

- ▶ 구조 정보를 입력 토큰 수준에서 명시 주입
- ▶ SaProt=결합 vocab · ProstT5=번역, 두 접근
- ▶ 원거리 상동성·구조 태스크 성능 ↑

### 한계 · 주의

- 3Di 얻으려면 사전 구조(예측/실험) 필요
- 이산화로 정보 손실 가능

## 임베딩이 담는 것 · 형태

### 창발하는 신호

- 2차구조 · 잔기 contact
- 보존 · 기능 부위
- 변이 효과(fitness) 경향
- 원격 상동성(기능 유사)

### 표현 형태

- per-residue:  $L \times d$  행렬(잔기 태스크)
- per-protein: mean-pool / CLS(전역)
- layer 선택도 성능에 영향
- 검색 · 클러스터링에 벡터 그대로

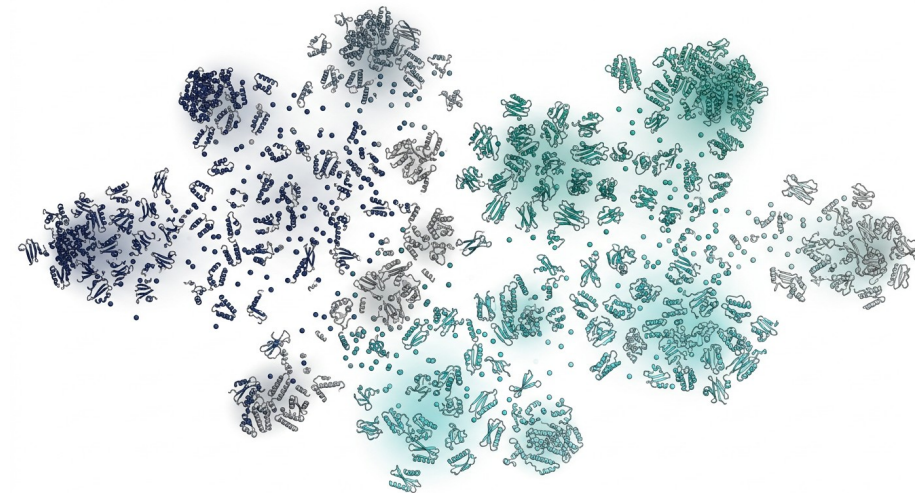


함정

pooling · layer 선택이 성능을 바꾼다 — 마지막 layer가 최선이 아닐 때 많고, mean-pool은 위치 정보를 잃는다

## 임베딩 = 단백질의 '좌표'

- 각 단백질 → 고정 차원 벡터(공간의 한 점)
- 가까운 벡터 = 유사 구조·기능
- 검색·클러스터링·유사체 탐색
- 원격 상동성(서열은 다르나 기능 유사) 포착



**+심화**

임베딩 공간에서의 거리 = 진화·기능 유사도의 근사 — BLAST가 놓치는 원격 관계를 임베딩이 찾기도(항상은 아님). PART E 실습에서 이 성질을 실제 20개 단백질로 확인한다.



## ESM-2 임베딩 추출 (fair-esm)

서열 → ESM-2 임베딩(1280차원)

```
import torch, esm
model, alphabet = esm.pretrained.esm2_t33_650M_UR50D() # dim 1280
bc = alphabet.get_batch_converter(); model.eval()
_,_,toks = bc([('p1', 'MKTAYIAKQR')])
with torch.no_grad(): out = model(toks, repr_layers=[33])
emb = out['representations'][33][0,1:-1] # [L,1280]
per_protein = emb.mean(0) # [1280] 벡터
```

▶ 이 벡터가 downstream(분류·회귀)의 입력 — 수작업 feature 대신 학습된 표현. (transformers API는 PART E 실습에서)



함정

BOS/EOS 토큰 제거 필수([1:-1]). 큰 모델·긴 서열은 GPU 메모리 부담 → 배치·truncation 관리

## 임베딩 활용 — pooling & 전이학습 전략

### pooling(태스크에 맞춤)

- 잔기 태스크 → per-residue( $L \times d$ )
- 전역 분류·국재화 → pooled( $d$ )
- 서열 검색 → per-protein 벡터 거리

### 전이학습 전략

- frozen probing: 임베딩 고정 + 얇은 분류기
- fine-tuning: 본체까지 미세조정
- LoRA/adapter: 저비용 부분 조정

#### +심화

태스크 granularity에 pooling을 맞추고, 데이터가 적으면 frozen probing부터. 최근엔 본체 고정 + adapter/LoRA로 저비용 fine-tuning이 실무 표준이 되고 있다.

## PART B

# 구조 기반 표현 (structure-based)

3D를 담는다 — contact · graph · surface

## 왜 구조를 표현에 담나

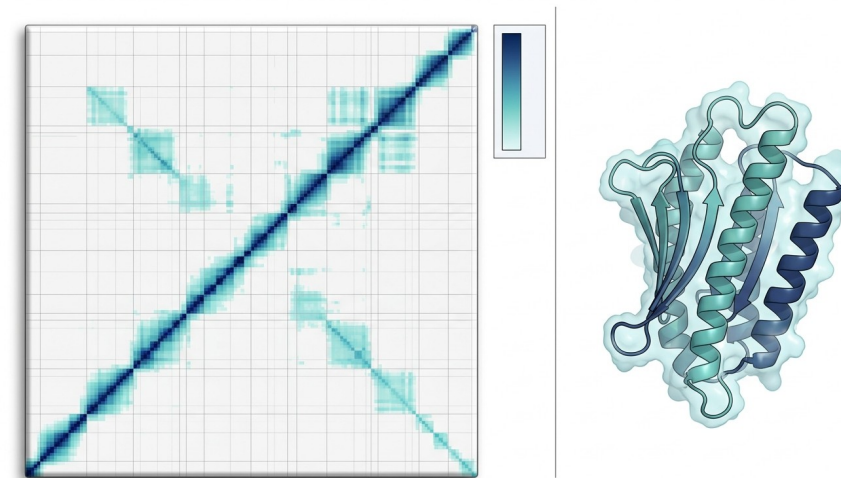
- 기능은 3D에서 나온다 — 활성부위·결합계면은 3D 배치로 결정(구조=기능, 1교시)
- 서열이 멀어도 3D에서 가까운 잔기가 함께 작동(서열 표현이 놓치는 관계)
- 결합부위·표면·단백질-단백질 상호작용(PPI)은 구조 표현이 유리
- AlphaFold DB로 구조가 대규모로 생겨 '구조 표현'을 실제로 쓸 수 있게 됨

**+심화**

단, 구조 표현의 품질은 '입력 구조의 품질'에 종속된다 — 예측 구조를 쓸 때는 신뢰도(pLDDT)를 반드시 함께 본다(PART C).

## contact / distance map

- 잔기 쌍이 3D에서 가까운지( $C\beta < 8\text{\AA}$ ) →  $L \times L$  행렬
- distance map: 실수 거리 행렬
- 2D 이미지처럼 CNN 처리 가능
- 구조를 잃지 않는 compact 표현 · 회전 불변



용어 · contact map

잔기 쌍의 근접 여부를 담은  $L \times L$  행렬 — 구조 정보를 2D로 압축한 표현

## 공진화 → contact — DCA · trRosetta · PLM attention

### 핵심 원리

1. **신호원** — 공진화: 함께 변하는 잔기쌍 = 3D 접촉 가능성
2. **DCA/Potts** — MSA에서 직접 커플링 추정 → 간접 상관 제거
3. **지도 딥러닝** — DCA+feature를 ResNet으로 → 거리·방향 분포(trRosetta)
4. **PLM attention** — ESM attention head 대칭화 → contact(비지도·MSA無)
5. **출력** —  $L \times L$  contact/distance map → 구조예측·표현 입력
6. **AF2 흡수** — Evoformer pair 표현이 contact를 내부 학습·활용

### 차별점 · 강점

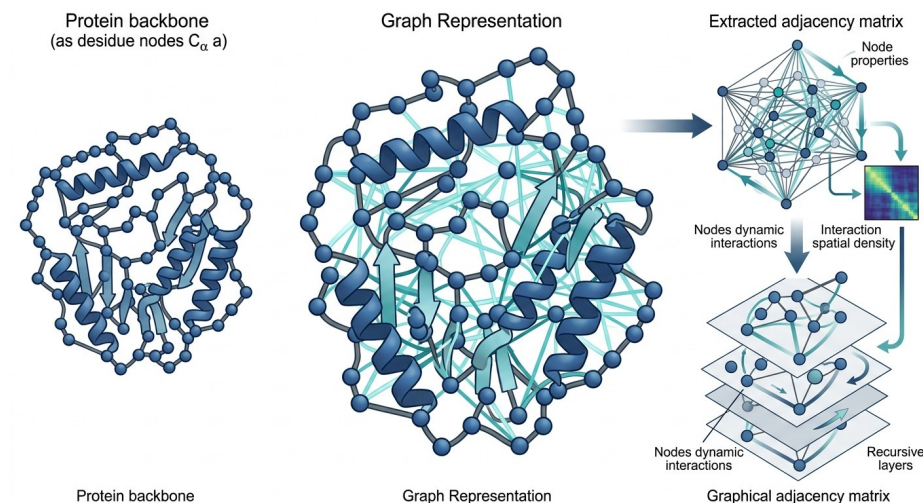
- ▶ 상관→직접커플링(DCA)로 간접신호 제거
- ▶ 딥러닝(trRosetta)으로 정확도 급상승
- ▶ PLM은 MSA 없이 attention으로 contact

### 한계 · 주의

- MSA 없으면 공진화 신호 약함
- 예측 map은 근사(구조≠확정)

## residue graph → GNN

- 노드=잔기, 엣지=공간/서열 근접
- 메시지 전달로 3D 관계 학습
- 결합부위·기능 예측에 강력
- GVP/GearNet: 회전 등변·자기지도 사전학습



용어 · residue graph

잔기를 노드로, 근접 관계를 엣지로 만든 그래프 — GNN 입력(Graphon 등)

# MaSIF / dMaSIF — 분자 표면 학습

## 핵심 원리

1. **표면 생성** — 분자 solvent-excluded surface를 삼각 메시로 계산
2. **패치 분해** — 각 점 주변 geodesic 반경의 중첩 패치로 분할
3. **기하 feature** — shape index · 거리의존 곡률(패치 형태)
4. **화학 feature** — 소수성 · 전하(Poisson-Boltzmann) · H-bond donor/acceptor
5. **geodesic conv** — 극좌표 패치에 학습 conv → 표면 패치 '지문'
6. **dMaSIF** — 메시 없이 원자 point cloud에서 즉석 · 미분가능 계산

## 차별점 · 강점

- ▶ 서열/골격이 아닌 '표면 기하+화학'으로 표현
- ▶ 상호작용(계면 · 리간드 부위)에 특화
- ▶ dMaSIF는 전처리 없이 end-to-end

## 한계 · 주의

- 표면(=구조) 품질에 의존
- 대형 표면 계산 비용



## backbone angle · voxel · point cloud

### 국소 · 격자

- 골격 이면각( $\phi/\psi/\omega$ )
- voxel/3D-CNN: 원자 밀도 격자
- 잔기·부위 주변 3D 환경

### 점군 · 등변

- 원자 좌표 point cloud
- 회전 불변/등변 처리 필요
- GVP-GNN: 3D 방향까지 담는 그래프

### 용어 · 등변성(equivariance)

입력 좌표를 회전하면 벡터 출력도 같이 회전 — 좌표계에 무관한 물리적으로 타당한 표현

# 구조 표현 — 한눈에 · 언제 유리한가

| 표현                   | 형태         | 강점 · 언제                   |
|----------------------|------------|---------------------------|
| contact/distance map | L×L 행렬     | 2D-CNN · compact, 구조예측 연계 |
| voxel / point cloud  | 3D 격자 · 점군 | 부위 · 원자 수준(도킹 · 포켓)       |
| residue graph(GNN)   | 노드 · 엣지    | 관계 · 결합부위 · 기능 예측         |
| surface(MaSIF)       | 표면 패치      | PPI · 리간드 결합부위 특화         |

+심화

공통 조건: 좋은 구조가 있어야 좋은 구조 표현 — 그래서 구조 예측(다음 PART C)이 표현의 '재료 공급'으로 결정적이다.

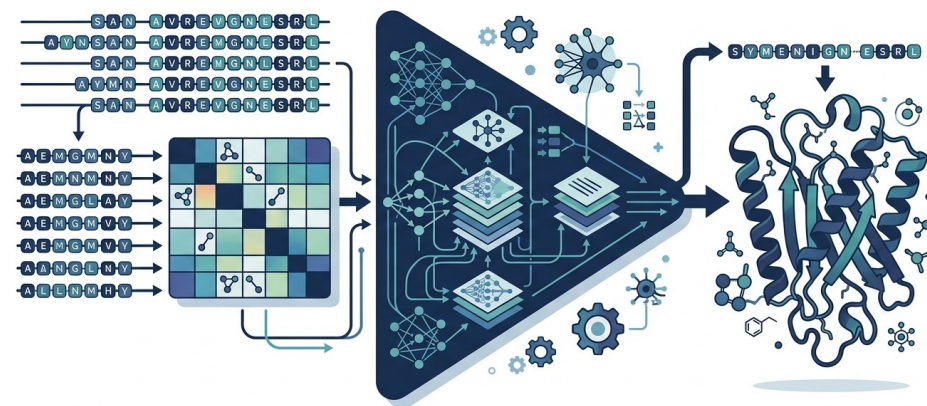
## PART C

# 구조 예측을 표현 소스로

AlphaFold · ESMFold가 구조를 공급한다

# AlphaFold2 — MSA·pair → 3D 구조

- 입력: 서열 + MSA(공진화) + 템플릿
- Evoformer: MSA·잔기쌍 표현 상호 정제
- 구조 모듈(IPA): 잔기 3D 프레임 직접 생성
- CASP14서 실험 정확도급( $GDT \approx 92$ )



용어 · Evoformer

MSA×pair 표현을 삼각 갱신으로 정제하는 AF2 핵심 트렁크 — 공진화를 구조로 변환

# AlphaFold2 — 구조 표현의 '재료 공급'

## 핵심 원리

1. **입력** — 표적 서열 + MSA(동족체) + 템플릿
2. **Evoformer(48블록)** — MSA 표현 & 잔기쌍(pair) 표현을 상호 갱신
3. **triangle update** — pair에 삼각부등식형 갱신 → 거리 일관성 강제
4. **구조 모듈(IPA)** — 각 잔기 3D 프레임(회전+평행이동) 직접 생성
5. **Recycling** — 출력을 재투입 정제 + pLDDT/PAE 자체 추정
6. **표현으로** — 예측 3D → contact·graph·surface 표현의 소스

## 차별점 · 강점

- ▶ end-to-end 학습으로 실험 정확도급 구조
- ▶ MSA 공변이를 pair 표현이 직접 활용
- ▶ 구조 표현을 대규모로 가능하게 함

## 한계 · 주의

- 단일 구조(앙상블·apo 경향)
- MSA 알르면 저하·점변이 약함
- 정밀 결합엔 실험구조 우위

## AF3 · Boltz · Chai — 복합체 · 오픈 생태계 (2024-25)

### 핵심 원리

1. **AlphaFold3** — Evoformer→Pairformer + diffusion으로 단백질 · 핵산 · 리간드 복합체 예측
2. **RoseTTAFold-AA** — 3-track을 all-atom · 임의 분자로 일반화(오픈, 2024)
3. **Boltz-1** — 오픈소스로 AlphaFold3급 정확도 재현(2024.11)
4. **Boltz-2** — 구조 + 결합친화도(affinity)까지 통합 예측(2025)
5. **Chai-1** — 실험 restraint 주입 시 정확도 ↑ · MSA-free 모드
6. **의미** — 복합체 구조도 '표현 소스' → 계면 · PPI 표현으로 확장

### 차별점 · 강점

- ▶ AF2(단백질)→AF3(범용 생체분자)
- ▶ 오픈 재현(Boltz) · 친화도(Boltz-2)까지
- ▶ 복합체 구조가 구조 표현의 재료로 편입

### 한계 · 주의

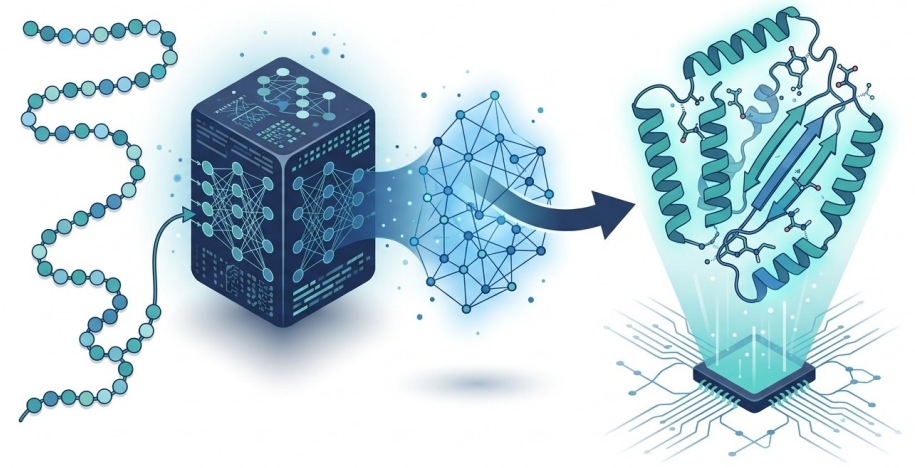
- 자체 벤치마크 주장(제3자 검증 권장)
- 친화도 예측은 본질적으로 난제

## 구조가 대규모로 생겼다

- AlphaFold DB: 2억(214M)+ 예측 구조
- ESM Metagenomic Atlas: 6억+ 구조
- → 실험구조 없는 단백질도 구조 표현 가능
- 구조 기반 표현(graph·surface)의 적용 범위가 폭발적으로 확장

# ESMFold — MSA 없이 단일서열 폴딩

- PLM(ESM-2) 임베딩 → folding trunk → 3D
- MSA 검색 불필요 → AF2 대비 ~수십배 빠름
- 고아·메타지놈 서열에 강점
- Metagenomic Atlas 6억+ 구조 생성



**+심화**

ESMFold = '임베딩에 구조가 담겨 있다'의 가장 강력한 증명 — 언어모델 표현만으로 구조를 직접 접는다(PART A와 연결). 단, 깊은 MSA 타깃엔 AF2가 여전히 우세.



# 구조를 어떻게 얻나 — 실무 경로

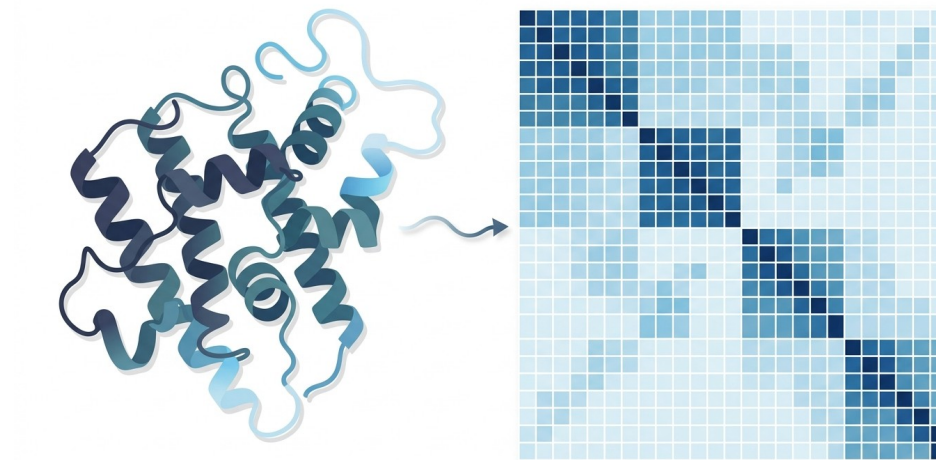
| 방법              | 특징                    | 용도        |
|-----------------|-----------------------|-----------|
| AlphaFold DB 조회 | 이미 계산됨 · 즉시           | 알려진 단백질   |
| ColabFold       | MMseqs2 MSA · 빠름 · 무료 | 신규 서열     |
| ESMFold         | 단일서열 · 초고속            | 대량 · 고아   |
| Boltz / AF3     | 복합체 · 리간드             | 도킹 · SBDD |

+심화

실무: 표현에 쓸 구조는 대개 AlphaFold DB 조회 또는 ColabFold/ESMFold로 생성 → 그 좌표로 graph/surface 표현을 만든다.

## pLDDT · PAE — 예측을 얼마나 믿나

- pLDDT: 잔기별 신뢰도(0-100)
- PAE: 잔기쌍 상대위치 오차( $L \times L$ )
- pTM/ipTM: 전역·계면 요약(복합체)
- 낮은 pLDDT = 무질서(disorder) 가능



용어 · pLDDT→PAE→pTM→ipTM

잔기 → 잔기쌍 → 전역 → 계면으로 이어지는 신뢰도 지표의 위계

# 예측을 얼마나 믿나 — pLDDT · PAE

| 지표          | 의미           | 해석     |
|-------------|--------------|--------|
| pLDDT >90   | 잔기 신뢰(매우 높음) | 좌표 신뢰  |
| pLDDT 70-90 | 신뢰(백본)       | 대체로 OK |
| pLDDT <50   | 낮음           | 무질서 가능 |
| PAE / ipTM  | 도메인 상대배치·계면  | 복합체 신뢰 |

+심화

pLDDT는 '좌표 신뢰도'이자 무질서 proxy — 구조 표현에 쓸 때 낮은 pLDDT 영역은 가중·제외 등 취급이 필요하다.

## 예측 구조의 한계 — 정직하게

- 하나의 구조(앙상블 아님) · 주로 apo 경향
- 점 돌연변이 · 유연성 · 리간드 유발 변화 약함
- 도킹 · 가상스크리닝엔 실험구조보다 약함  
Karelina 2023 · Wong 2022
- → 표현 소스로 유용하나 '맹신 금지'



함정

AlphaFold 구조를 도킹 · 정밀 결합 분석에 그대로 쓰면 실패 가능 — fold · pocket은 좋아도 holo · 정밀 배치는 별개

# 구조 표현 → 구조 기반 약물설계(SBDD)

## 구조가 여는 것

- 예측 구조로 결합 포켓·활성부위 파악
- 표면·graph 표현이 리간드·PPI 부위 강조
- 복합체(co-folding)로 계면 예측

## 설계·스크리닝 연결

- 포켓 → 도킹·가상 스크리닝
- 구조 기반 생성 설계(바인더·항체)
- 표현이 예측을 넘어 '설계'로 확장

### +심화

구조 예측·구조 표현은 SBDD의 입구 — 예측 구조의 pocket으로 도킹/설계를 시작한다. 단, 정밀 결합·랭킹은 실험 검증 필요(앞 슬라이드 한계와 함께 읽을 것).

PART D

# 비교 · 선택

무엇을 언제 쓰나 — 정직한 평가

# 표현법 선택 — 4가지 축

| 상황        | 권장 표현                | 이유        |
|-----------|----------------------|-----------|
| 데이터 매우 적음 | 물성·조성(저차원)           | 과적합 방지·해석 |
| 중간·해석 필요  | 조성/PSSM + 물성         | 투명·근거 제시  |
| 데이터 많음·복잡 | PLM 임베딩(ESM/ProtT5)  | 문맥·전이·성능  |
| 결합·계면·PPI | 구조 표현(graph/surface) | 3D 배치가 본질 |
| 계산자원 제약   | 조성·물성 or 소형 PLM      | 임베딩 비용 ↓  |

+심화

판단 4축: ① 데이터량 ② 과제(분류/회귀/생성/상호작용) ③ 해석성 요구 ④ 계산비용. 실무 시작점은 '강한 baseline(조성/물성) + PLM 임베딩'을 함께 돌려 비교하는 것.

## 과제 유형별 표현 매칭

분류 (예: 국재화·기능 라벨)

per-protein 임베딩(mean-pool) 또는 조성 + 얇은 분류기

회귀 (예: 안정성·용해도 점수)

PLM 임베딩 + 물성 하이브리드, 소데이터면 물성 우선

잔기 단위 (예: 2차구조·contact)

per-residue 임베딩( $L \times d$ ) 또는 PSSM 프로파일

상호작용·결합 (PPI·리간드)

구조 표현(graph/surface) + 임베딩 결합

생성·설계

자기회귀/멀티모달 PLM(ProGen2·ESM-3 계열)



# 표현을 '공정하게' 비교하려면

| 벤치마크       | 무엇을 측정                           | 메모      |
|------------|----------------------------------|---------|
| TAPE       | 2차구조 · contact · stability 등 5과제 | 프레임 정립  |
| PEER       | 기능 · 국재화 · 구조 · 상호작용 다과제         | 멀티태스크   |
| FLIP       | 단백질 엔지니어링 현실 split               | fitness |
| ProteinGym | 250+ DMS · 수백만 변이                | 변이효과    |

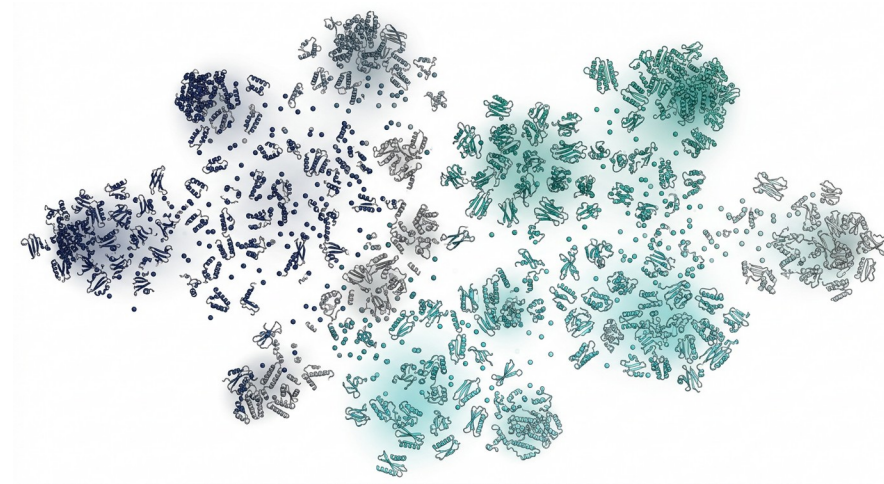


함정

구체적 정확도 · 상관계수 등 성능 수치는 출처 · 조건 없이 인용 금지 — 본 자료는 정성 비교만(수치는 원논문 · 벤치마크 확인 권장)

## PLM이 항상 이기나? — 정직한 답

- 구조·보존 연관 과제(2차구조·contact·변이효과)엔 강함
- 일부 엔지니어링/fitness 과제엔 단순 baseline이 대등
- TAPE: 사전학습이 정렬 feature에 못 미친 경우도
- → 반드시 baseline과 비교 (Shanehsazzadeh 2020)



⚠ 함정

PLM 임베딩이 무조건 낫다는 가정 금지 — fluorescence 등에서선 선형회귀가 이긴 사례도. 과제·데이터 의존이 큼

# 표현법의 공통 한계·주의

## 일반화·편향

- 도메인 외(신규 폴드·비천연) 일반화 저하
- 학습 데이터 편향(UniProt 편중)이 표현에 전이
- 고아·설계 단백질은 진화정보 표현 약함

## 실무 주의

- PLM 임베딩은 블랙박스 → 해석 어려움
- 구조 표현은 입력 구조 품질에 종속(pLDDT 확인)
- 계산비용·재현성(모델·layer·pooling 기록)

### +심화

어떤 표현도 만능이 아니다 — 과제·데이터·자원에 맞춰 고르고, baseline과 비교하고, 예측은 실험으로 확인한다는 원칙이 표현법 활용의 핵심.

PART E

# 오늘의 실습

20개 단백질 · 4 family로 표현을 직접 비교

## 오늘의 실습 — 표현법 직접 비교

- 노트북: 20260824/notebooks/protein\_representation\_practical.ipynb
- 실제 UniProt 단백질 20개(4개 family)를 실행 시 fetch
- 수작업 표현(2교시): one-hot · 조성(AAC) · 물성(ProtParam) 재사용
- 학습된 표현(3교시): ESM-2 임베딩(esm2\_t6\_8M, 320차원)
- 같은 과제(family 구분)에서 어느 표현이 나은지 실측·시각화

**+심화**

실습 원칙 — 무-날조: 서열은 실제 UniProt에서 내려받고, 모든 성능 수치는 노트북이 실제로 계산. 20개 소규모 데모(정식 벤치마크 아님).

## 실습 데이터 — 4 family × 5 (UniProt)

| family          | 예시                                         | 특징       |
|-----------------|--------------------------------------------|----------|
| Globin          | 헤모글로빈 $\alpha/\beta/\delta/\gamma$ · 미오글로빈 | 산소 운반    |
| Serine protease | 트립신 · 키모트립시노겐 · 엘라스타제 · PSA                | 촉매(가수분해) |
| Protein kinase  | SRC · CDK1/2 · ERK2 · PKC $\delta$         | 인산화 신호   |
| Cytochrome P450 | CYP3A4/2D6/2C9/1A2/2B6                     | 약물 대사    |

**+심화**

기능군이 뚜렷이 다른 4 family → '표현이 family를 구분하는지'를 보기 좋은 세팅. 라벨은 실제 UniProt 분류.

## 표현 만들기 — 수작업 vs ESM-2

서열 → AAC·물성(2교시) & ESM-2 임베딩(3교시)

```
# 수작업 표현(2교시 재사용): AAC 20d · 물성 7d
X_aac = np.vstack([aac(s)      for s in df['seq']])
X_phys = np.vstack([physchem(s) for s in df['seq']])
# 학습된 표현(3교시): ESM-2 임베딩 (transformers)
from transformers import AutoTokenizer, AutoModel
tok = AutoTokenizer.from_pretrained('facebook/esm2_t6_8M_UR50D')
esm = AutoModel.from_pretrained('facebook/esm2_t6_8M_UR50D').eval()
out = esm(**tok(seq, return_tensors='pt')).last_hidden_state[0]
emb = out[1:-1].mean(0) # 특수토큰 제외 mean-pool → 320d
```

▶ 2교시 수작업 표현과 3교시 ESM-2 임베딩을 같은 20개 단백질에 대해 나란히 계산

## 공정 비교 — LOO 1-NN & 실루엣

같은 과제(family)에서 표현별 성능 실측

```
from sklearn.model_selection import LeaveOneOut, cross_val_predict
from sklearn.metrics import accuracy_score, silhouette_score
reps = {'AAC': X_aac, '물성': X_phys, 'ESM-2': X_esm}
for name, X in reps.items():
    Xs = StandardScaler().fit_transform(X)
    pred = cross_val_predict(KNeighborsClassifier(1), Xs, y, cv=LeaveOneOut())
    acc = accuracy_score(y, pred)      # family 분류 정확도
    sil = silhouette_score(Xs, y)      # 군집 분리도
```

▶ LOO(각 단백질을 하나씩 빼고 1-NN으로 family 예측) + 실루엣(같은 family끼리 뭉치는 정도)



# 표현별 family 구분 성능 (실측치)

| 표현                       | LOO 1-NN family 정확도 | 실루엣(군집 분리도) |
|--------------------------|---------------------|-------------|
| AAC (조성, 20d) — 2교시      | 0.70                | 0.13        |
| 물성 (ProtParam, 7d) — 2교시 | 0.95                | 0.21        |
| ESM-2 (임베딩, 320d) — 3교시  | 1.00                | 0.50        |

→ 학습된 표현(ESM-2)이 family를 가장 잘 구분: 정확도 1.00 · 실루엣 0.50 (수작업 표현보다 뚜렷이 높음)

+심화

이 수치는 노트북(esm2\_t6\_8M)의 실제 계산값 — 지어낸 값이 아님. 단, 20개 소규모 데모이므로 값은 모델·전처리·실행에 따라 달라질 수 있다(정식 결론 아님).

## 실측이 말해주는 것

- **정확도 순서: AAC(0.70) < 물성(0.95) < ESM-2(1.00)**  
순서·문맥을 담을수록 family 구분 ↑
- **실루엣 순서: 0.13 < 0.21 < 0.50**  
ESM-2 임베딩이 family끼리 가장 뚜렷이 뭉침
- **수작업 표현도 유용 — 물성 7차원만으로 0.95**  
해석 가능·저비용의 강한 baseline
- **PLM이 항상 이기지는 않음(PART D) — 여기선 family 구분에 유리**

**+심화**

교훈: '모델을 바꾸기 전에 표현을 먼저 의심하라.' 같은 20개 단백질·같은 분류기라도, 어떤 벡터로 바꾸느냐가 결과(0.70→1.00)를 가른다. 단, 소규모 데모 단서 유지.

## 생성모델이 만드는 표현 — 잠재공간(latent space)

### 판별(읽기) — 3교시까지

- 서열/구조 → 표현(임베딩) → 예측
- ESM-2 · 구조그래프로 family · 위치 · 물성
- 표현은 분석의 '입력'

### 생성(만들기) — RFdiffusion · ProteinMPNN

- 잠재공간에서 새 단백질을 샘플링
- RFdiffusion: 구조를 노이즈 → 디노이즈(확산)
- ProteinMPNN: 구조표현 → 서열(inverse folding)

#### +심화

핵심: 같은 '표현'이 판별의 입력이자 생성의 좌표계다. RFdiffusion은 구조의 확산 잠재과정을, ProteinMPNN은 구조 그래프 표현을 디코딩해 서열을 만든다. PLM 임베딩(3교시)이 판별·생성 양쪽의 공통 기반.

## 표현의 순환 — 설계와 검증 (오늘 실습 NB3)

- **RFdiffusion:** 조건(모티프·대칭·결합타겟) → 백본 구조 생성  
구조 표현을 '생성'
- **ProteinMPNN:** 생성 백본 → 잔기별 서열 설계  
구조 표현 → 서열
- **ESMFold / Boltz-2:** 설계 서열 → 재접힘  
서열 → 구조로 되돌림
- **self-consistency:** 재접힌 구조가 원 설계와 일치하나?  
표현이 순환하며 스스로 검증

**+심화**

표현은 예측의 입력을 넘어 '설계의 언어'다: 구조↔서열↔임베딩을 오가며 새 단백질을 만들고 되돌려 확인한다. 생성 결과는 가설 — 합성·assay 실험 검증이 뒤따라야 한다.

## 7/24 전체 종합 — 1 → 2 → 3교시

### 1교시

구조생물학 기초 — 아미노산·1~4차 구조·폴딩(구조=기능)

### 2교시

수작업 표현 — one-hot·조성·물성(ProtParam)·PSSM·예측 툴

### 3교시

학습된 표현(PLM) & 구조 기반 표현 → 비교·실습

**구조 → 수작업 표현 → 학습·구조 표현. 단백질을 '어떻게 표현하는가'의 전 스펙트럼.**  
표현이 파이프라인의 맨 앞이자 성능의 상한 — 과제·데이터로 고르고 baseline과 비교, 예측은 실험으로.

## 3교시 — 핵심 정리

1

**PLM(ESM·ProtT5)이 서열에서 표현을 '학습'**

MLM 사전학습 → 구조·기능·contact 창발

2

**임베딩을 전이학습에 활용**

per-residue/per-protein · frozen/fine-tune/LoRA

3

**구조 표현: contact·graph·surface·voxel**

3D 배치가 본질인 과제에 유리

4

**AlphaFold/ESMFold가 구조를 대량 공급**

단, pLDDT·양상불·정밀결합 한계 정직하게

5

**실측: 표현이 결과를 가르다(0.70→1.00)**

만능 없음 — 과제·데이터로 선택+baseline 비교

# 감사합니다

3교시 · 학습된 표현 & 구조 표현 — 표현이 downstream 성능을 좌우한다

한 문장 요약 — '모델을 바꾸기 전에 표현을 먼저 의심하라.'

같은 단백질도 어떤 벡터로 바꾸느냐가 결과를 가른다(실습 실측: AAC 0.70 → ESM-2 1.00).

# 주요 출처 (3교시)

• PLM: Rives 2021 (ESM-1b, PNAS); Lin 2023 (ESM-2/ESMFold, Science); Elnaggar 2022 (ProtTrans/ProtT5, IEEE TPAMI); Rao 2021 (MSA Transformer, ICML).

• PLM(계열): Elnaggar 2023 (Ankh); Nijkamp 2023 (ProGen2); ESM-C 2024 (기술공개, peer-review 전); Hayes 2025 (ESM-3, Science 387:850).

• 사전학습 기반: Devlin 2019 (BERT); Rao 2019 (TAPE, NeurIPS).

• 구조 표현: Gainza 2020 (MaSIF, Nat Methods); Sverrisson 2021 (dMaSIF, CVPR); Jing 2021 (GVP, ICLR); Zhang 2023 (GearNet, ICLR).

• contact/공진화: Morcos 2011 (DCA); Yang 2020 (trRosetta, PNAS); Rao 2021 (ESM contact).

• 구조 예측: Jumper 2021 (AF2, Nature); Abramson 2024 (AF3, Nature 630:493); Lin 2023 (ESMFold); Mirdita 2022 (ColabFold, Nat Methods); Varadi 2024 (AlphaFold DB, NAR).

• 설계·접점: Watson 2023 (RFDiffusion, Nature); Dauparas 2022 (ProteinMPNN, Science).

• 벤치마크·한계: Xu 2022 (PEER, NeurIPS); Notin 2023 (ProteinGym, NeurIPS); Dallago 2021 (FLIP); Shanehsazzadeh 2020; Karelina 2023 (eLife); Wong 2022 (Mol Syst Biol).

• 실습 도구: Cock 2009 (Biopython, Bioinformatics); scikit-learn; UniProt. 실측치(0.70/0.95/1.00·실루엣 0.13/0.21/0.50)는 본 노트북 20개 데모 계산값.